

UNIFEOB
CENTRO UNIVERSITÁRIO DA FUNDAÇÃO DE ENSINO OCTÁVIO BASTOS
ESCOLA DE NEGÓCIOS

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
CIÊNCIAS DA COMPUTAÇÃO

CRUD de Todas as Entidades, Validações e Testes da API no Insomnia –
Entrega Prof. Nivaldo Andrade 06/10/2026

ESTUDANTES:

25000151 - Guilherme P. da Rosa Santi
25001176 - Henrique de Oliveira Molinari
25000019 - João Augusto de Freitas
25000795 - Kauan Leander Leandrini

São João da Boa Vista–SP
2026

1. Contexto e objetivo

Esta entrega atende à atividade do projeto de extensão proposta em 29/09/2026 pelo Prof. Nivaldo Andrade: **implementação das funcionalidades CRUD, validações básicas e tratamento de erros e testes das rotas com o Insomnia**. Ela foi entregue em 06/10/2026.

A entrega anterior (15/09/2026) cobriu o CRUD de uma entidade, ferramentas. Desta vez a coleção do Insomnia cobre **todas as entidades do banco que a API expõe**: setores, categorias, atividades, colaboradores, ferramentas, empréstimos, ocorrências, notificações e feriados, além do convite de acesso (tabelas *convites_acesso* e *usuarios*). Para cada entidade há cinco requisições, uma por verbo: **POST** (criar), **GET** (consultar), **PUT** (substituir), **PATCH** (editar) e **DELETE** (dar baixa).

Nem toda entidade aceita os cinco verbos, e isso é proposital: o empréstimo nunca é apagado, a ocorrência nasce da devolução com avaria, a notificação é gerada pelo sistema. Nesses casos a requisição existe e prova que a rota **responde 404** em vez de aceitar a operação, e a explicação está no texto da própria requisição. A seção 3 traz a matriz completa.

No total são **71 requisições** em 12 pastas, com **176 testes automáticos**. Na execução registrada neste documento, **71 de 71 requisições passaram em todos os testes (176 de 176 testes)**.

Os arquivos que sustentam a entrega estão no repositório do projeto:

- *docs/insomnia/entrega-nivaldo-06.10.2026.json*: coleção do Insomnia (71 requisições, cada uma com documentação e testes dentro);
- *docs/entregas-pi/tecnologias-desenvolvimento-web-nivaldo-andrade/entrega-nivaldo-06.10.2026.pdf*: este documento.

2. Ambiente e forma de execução

2.1 Ambiente

Os testes rodaram em ambiente local, contra o banco de desenvolvimento, com a API Node.js/Express (TypeScript) na porta 3000 e o PostgreSQL com o banco *soufer_dev* populado pelo seed. O cliente HTTP é o Insomnia 13.1. A API passa também na suíte automática do projeto (Vitest + Supertest): 31 arquivos e 320 testes aprovados na data da entrega.

```
cd api
npm run db:migrate # cria as tabelas
npm run db:seed   # popula usuários, colaboradores e ferramentas
npm run dev       # API em http://localhost:3000
```

Credenciais de teste criadas pelo seed: admin, matrícula **0053**, e manutenção, matrícula **0001**, ambos com a senha **123456**.

As notificações (pasta 9) são geradas por uma rotina diária do banco (*fn_gerar_notificacoes()*, seg a sex às 07:00). Para que o teste de "marcar como lida" tenha uma notificação real, a função foi executada uma vez no banco de teste antes da coleção. Sem ela a listagem vem vazia e o teste cobre o caso de id inexistente (404).

2.2 Como importar e executar

- No Insomnia, use **Import** e escolha *docs/insomnia/entrega-nivaldo-06.10.2026.json*.
- Abra a coleção e rode as 12 pastas **na ordem**, pelo Collection Runner ou uma a uma. A pasta 1 faz o login e cria os dados de apoio; a pasta 12 dá baixa neles.
- Cada requisição traz na aba **Docs** o que faz, por que existe, como está configurada e o resultado esperado, e na aba **Scripts** → **After-response** os testes automáticos.

2.3 Variáveis de ambiente

A coleção não copia token nem ids à mão. Cada requisição que produz um dado o grava em uma variável do Base Environment por meio de um script pós-resposta, e as requisições seguintes o leem (`{{ _.token }}`, `{{ _.setor_id }}` etc.). As variáveis começam vazias, exceto *base_url*, e não devem ser apagadas nem renomeadas.

Variável	Quem grava	Quem usa
<i>base_url</i>	já vem preenchida	todas (endereço da API, <code>http://localhost:3000/v1</code>)
<i>token</i>	01 Login do admin	todas as rotas protegidas
<i>token_manut</i>	02 Login da manutenção	59, 60 (testes de 403)
<i>sufixo, hoje, matricula_a, matricula_b</i>	01 Login do admin (script)	nomes únicos, previsão de devolução e matrículas dos colaboradores de teste
<i>setor_apoio_id, grupo_apoio_id, atividade_apoio_id</i>	03 Obter ids (auxiliar)	cadastros de colaborador, ferramenta e empréstimo
<i>colaborador_apoio_id, ferramenta_apoio1_id, ferramenta_apoio2_id</i>	04, 05 e 06 (dados de apoio)	pastas 7, 8, 11 e 12
<i>setor_id, categoria_id, atividade_id</i>	07, 12 e 17 (POST de cada cadastro)	GET, PUT, PATCH e DELETE da própria pasta
<i>colaborador_id, token_convite</i>	22 e 26	pasta 5
<i>ferramenta_id</i>	30	pasta 6
<i>emprestimo_id, emprestimo2_id</i>	35 e 41	pastas 7 e 8
<i>ocorrencia_id, notificacao_id</i>	43 e 48	pastas 8 e 9

3. Cobertura: entidades do banco e verbos

A tabela mostra, para cada entidade, qual requisição exercita cada verbo. **Verde (✓)**: a rota existe e foi testada com sucesso. **Amarelo (404)**: a API **não oferece** o verbo por regra de negócio e a requisição prova que a rota responde 404 ("Rota não encontrada").

Entidade (tabela)	POST	GET	PUT	PATCH	DELETE
Setores setores	✓ req. 07	✓ req. 08	✓ req. 09	✓ req. 10	✓ req. 11
Categorias grupos_ferramentas	✓ req. 12	✓ req. 13	✓ req. 14	✓ req. 15	✓ req. 16
Atividades atividades	✓ req. 17	✓ req. 18	✓ req. 19	✓ req. 20	✓ req. 21
Colaboradores colaboradores	✓ req. 22	✓ req. 23	404 req. 24	✓ req. 25	✓ req. 29
Ferramentas ferramentas	✓ req. 30	✓ req. 31	✓ req. 32	✓ req. 33	✓ req. 34
Empréstimos emprestimos	✓ req. 35	✓ req. 36	404 req. 37	✓ req. 38	404 req. 39
Ocorrências ocorrencias	404 req. 40	✓ req. 43	404 req. 44	✓ req. 45	404 req. 46
Notificações notificacoes	404 req. 47	✓ req. 48	404 req. 49	✓ req. 50	404 req. 51
Feriados feriados	✓ req. 52	✓ req. 53	404 req. 54	404 req. 55	404 req. 56

3.1 Como cada verbo foi interpretado

- **POST** cria o registro. Em ocorrências o "POST" acontece por dentro da API, na devolução com avaria (req. 42); nos feriados é a sincronização com a BrasilAPI (*POST /feriados/sincronizar*).
- **GET** consulta por id ou por lista com filtros e paginação. Empréstimos, ocorrências, notificações e feriados são consultados pela lista.
- **PUT** substitui o recurso. Em setores, categorias e atividades o PUT envia a representação completa. Em ferramentas, o PUT é o envio da foto (*PUT /ferramentas/:id/foto*), que substitui a imagem inteira, o uso correto do verbo. Nas demais entidades não há PUT, e a requisição documenta isso.
- **PATCH** altera só os campos enviados. Na devolução de empréstimo e na evolução de ocorrência e de notificação, o PATCH é o verbo da transição de estado.
- **DELETE** é **baixa lógica** (*ativo = false*) em setores, categorias, atividades, colaboradores e ferramentas: o registro continua no banco para preservar o histórico. Empréstimo, ocorrência, notificação e feriado não são apagados.

3.2 Demais tabelas do banco

Tabela	Como é exercitada
usuarios	Não tem CRUD próprio: a conta nasce quando o colaborador aceita o convite e define a senha (req. 28). O login é testado nas req. 01 e 02.

convites_acesso	Gerar o convite (req. 26), consultá-lo pelo link público (req. 27) e consumi-lo (req. 28).
subgrupos_ferramentas, itens_kit	Não têm rota própria na API atual; são referenciadas só como campos opcionais do cadastro de ferramenta. Não há o que testar por HTTP.
auditoria	É gravada pela própria API (por exemplo, ao disponibilizar uma ferramenta). Não tem rota de leitura nem de escrita.

4. Resultado da execução

Cada linha é uma requisição da coleção, na ordem de execução. O tempo é o da execução registrada. Todas as requisições passaram em todos os seus testes (coluna Testes).

Nº	Rota	Cenário	Resultado	Testes
01	<i>POST /v1/auth/login</i>	Login do admin	✓ 200	3/3
02	<i>POST /v1/auth/login</i>	Login da manutenção	✓ 200	3/3
03	<i>GET /v1/opcoes</i>	Obter ids de setor, categoria e atividade (auxiliar)	✓ 200	2/2
04	<i>POST /v1/colaboradores</i>	Criar colaborador de apoio (POST)	✓ 201	2/2
05	<i>POST /v1/ferramentas</i>	Criar ferramenta de apoio 1 (POST)	✓ 201	2/2
06	<i>POST /v1/ferramentas</i>	Criar ferramenta de apoio 2 (POST)	✓ 201	2/2
07	<i>POST /v1/setores</i>	Cadastrar setor (POST)	✓ 201	2/2
08	<i>GET /v1/setores/:setor_id</i>	Consultar setor por id (GET)	✓ 200	2/2
09	<i>PUT /v1/setores/:setor_id</i>	Substituir setor (PUT)	✓ 200	3/3

10	<i>PATCH /v1/setores/:setor_id</i>	Editar setor (PATCH, campo parcial)	✓ 200	3/3
11	<i>DELETE /v1/setores/:setor_id</i>	Dar baixa no setor (DELETE, baixa lógica)	✓ 200	2/2
12	<i>POST /v1/categorias</i>	Cadastrar categoria (POST)	✓ 201	2/2
13	<i>GET /v1/categorias/:categoria_id</i>	Consultar categoria por id (GET)	✓ 200	2/2
14	<i>PUT /v1/categorias/:categoria_id</i>	Substituir categoria (PUT)	✓ 200	3/3
15	<i>PATCH /v1/categorias/:categoria_id</i>	Editar categoria (PATCH, campo parcial)	✓ 200	3/3
16	<i>DELETE /v1/categorias/:categoria_id</i>	Dar baixa na categoria (DELETE, baixa lógica)	✓ 200	2/2
17	<i>POST /v1/atividades</i>	Cadastrar atividade (POST)	✓ 201	2/2
18	<i>GET /v1/atividades/:atividade_id</i>	Consultar atividade por id (GET)	✓ 200	2/2
19	<i>PUT /v1/atividades/:atividade_id</i>	Substituir atividade (PUT)	✓ 200	3/3
20	<i>PATCH /v1/atividades/:atividade_id</i>	Editar atividade (PATCH, campo parcial)	✓ 200	3/3
21	<i>DELETE /v1/atividades/:atividade_id</i>	Dar baixa na atividade (DELETE, baixa lógica)	✓ 200	2/2

22	POST /v1/colaboradores	Cadastrar colaborador (POST)	✓ 201	2/2
23	GET /v1/colaboradores/:colaborador_id	Consultar colaborador por id (GET)	✓ 200	2/2
24	PUT /v1/colaboradores/:colaborador_id	Substituir colaborador (PUT) — rota inexistente	X 404 NOT_FOUND	3/3
25	PATCH /v1/colaboradores/:colaborador_id	Editar colaborador (PATCH, campo parcial)	✓ 200	3/3
26	POST /v1/colaboradores/:colaborador_id/convite	Gerar convite de acesso (POST)	✓ 201	2/2
27	GET /v1/auth/convites/:token_convite	Consultar convite (GET, rota pública)	✓ 200	2/2
28	POST /v1/auth/convites/:token_convite/senha	Aceitar convite e definir senha (POST, cria usuário)	✓ 200	2/2
29	DELETE /v1/colaboradores/:colaborador_id	Inativar colaborador (DELETE, baixa lógica)	✓ 200	2/2
30	POST /v1/ferramentas	Cadastrar ferramenta (POST)	✓ 201	2/2
31	GET /v1/ferramentas/:ferramenta_id	Consultar ferramenta por id (GET)	✓ 200	2/2
32	PUT /v1/ferramentas/:ferramenta_id/foto	Enviar foto da ferramenta (PUT /foto)	✓ 200	2/2

33	PATCH /v1/ferramentas/:ferramenta_id	Editar ferramenta (PATCH, campos parciais)	✓ 200	3/3
34	DELETE /v1/ferramentas/:ferramenta_id	Dar baixa na ferramenta (DELETE, baixa lógica)	✓ 200	2/2
35	POST /v1/emprestimos	Registrar retirada (POST)	✓ 201	2/2
36	GET /v1/emprestimos	Listar empréstimos em aberto (GET)	✓ 200	2/2
37	PUT /v1/emprestimos/:emprestimo_id	Substituir empréstimo (PUT) — rota inexistente	X 404 NOT_FOUND	3/3
38	PATCH /v1/emprestimos/:emprestimo_id/devolucao	Registrar devolução (PATCH /devolucao)	✓ 200	2/2
39	DELETE /v1/emprestimos/:emprestimo_id	Apagar empréstimo (DELETE) — rota inexistente	X 404 NOT_FOUND	3/3
40	POST /v1/ocorrencias	Criar ocorrência direto (POST) — rota inexistente	X 404 NOT_FOUND	3/3
41	POST /v1/emprestimos	Retirar a ferramenta de apoio 2 (auxiliar)	✓ 201	2/2
42	PATCH /v1/emprestimos/:emprestimo2_id/devolucao	Devolver com avaria — abre a ocorrência (PATCH)	✓ 200	2/2

43	<i>GET /v1/ocorrencias</i>	Listar ocorrências abertas (GET)	✓ 200	2/2
44	<i>PUT /v1/ocorrencias/:ocorrencia_id</i>	Substituir ocorrência (PUT) — rota inexistente	X 404 NOT_FOUND	3/3
45	<i>PATCH /v1/ocorrencias/:ocorrencia_id</i>	Atualizar ocorrência (PATCH)	✓ 200	2/2
46	<i>DELETE /v1/ocorrencias/:ocorrencia_id</i>	Apagar ocorrência (DELETE) — rota inexistente	X 404 NOT_FOUND	3/3
47	<i>POST /v1/notificacoes</i>	Criar notificação (POST) — rota inexistente	X 404 NOT_FOUND	3/3
48	<i>GET /v1/notificacoes</i>	Listar notificações (GET)	✓ 200	2/2
49	<i>PUT /v1/notificacoes/:notificacao_id</i>	Substituir notificação (PUT) — rota inexistente	X 404 NOT_FOUND	3/3
50	<i>PATCH /v1/notificacoes/:notificacao_id/lida</i>	Marcar notificação como lida (PATCH)	✓ 200	2/2
51	<i>DELETE /v1/notificacoes/:notificacao_id</i>	Apagar notificação (DELETE) — rota inexistente	X 404 NOT_FOUND	3/3
52	<i>POST /v1/feriados/sincronizar</i>	Sincronizar feriados com a BrasilAPI (POST)	✓ 200	2/2
53	<i>GET /v1/feriados</i>	Consultar feriados do ano (GET)	✓ 200	2/2

54	<i>PUT /v1/feriados/1</i>	Substituir feriado (PUT) — rota inexistente	X 404 NOT_FOUND	3/3
55	<i>PATCH /v1/feriados/1</i>	Editar feriado (PATCH) — rota inexistente	X 404 NOT_FOUND	3/3
56	<i>DELETE /v1/feriados/1</i>	Apagar feriado (DELETE) — rota inexistente	X 404 NOT_FOUND	3/3
57	<i>POST /v1/setores</i>	Criar setor sem token — 401	X 401 TOKEN_NOT_PROVIDED	3/3
58	<i>POST /v1/setores</i>	Criar setor com token inválido — 401	X 401 TOKEN_INVALID	3/3
59	<i>POST /v1/setores</i>	Criar setor como manutenção — 403	X 403 ACCESS_DENIED	3/3
60	<i>PATCH /v1/colaboradores/:colaborador_apoio_id</i>	Editar colaborador como manutenção — 403	X 403 ACCESS_DENIED	3/3
61	<i>POST /v1/setores</i>	Criar setor com nome vazio — 400	X 400 VALIDATION_ERROR	3/3
62	<i>POST /v1/ferramentas</i>	Criar ferramenta sem grupold — 400	X 400 VALIDATION_ERROR	3/3
63	<i>POST /v1/colaboradores</i>	Criar colaborador com matrícula inválida — 400	X 400 VALIDATION_ERROR	3/3

64	<i>GET /v1/setores/abc</i>	Buscar setor com id não numérico — 400	X 400 VALIDATION_ERROR	3/3
65	<i>GET /v1/setores/999999</i>	Buscar setor inexistente — 404	X 404 NOT_FOUND	3/3
66	<i>GET /v1/ferramentas/999999</i>	Buscar ferramenta inexistente — 404	X 404 FERRAMENTA_NOT_FOUND	3/3
67	<i>POST /v1/setores</i>	Criar setor com nome repetido — 409	X 409 DUPLICATE_ENTRY	3/3
68	<i>POST /v1/colaboradores</i>	Criar colaborador com matrícula repetida — 409	X 409 DUPLICATE_ENTRY	3/3
69	<i>DELETE /v1/ferramentas/:ferramenta_apoio1_id</i>	Dar baixa na ferramenta de apoio 1	✓ 200	2/2
70	<i>DELETE /v1/ferramentas/:ferramenta_apoio2_id</i>	Dar baixa na ferramenta de apoio 2	✓ 200	2/2
71	<i>DELETE /v1/colaboradores/:colaborador_apoio_id</i>	Inativar o colaborador de apoio	✓ 200	2/2

5. Requisições, uma a uma

Para cada requisição: o que faz, por que existe, como está configurada, a requisição enviada e a resposta recebida na execução registrada, e os testes automáticos que a validam. Tokens de acesso aparecem abreviados.

5.1 1 · Autenticação e dados de apoio

Roda primeiro. Faz o login do admin e da manutenção, descobre ids de cadastros que já existem (setor, categoria e atividade) e cria os dados de apoio (um colaborador e duas ferramentas) que as pastas de Empréstimos e Ocorrências consomem. Os dados de apoio são removidos na pasta 12 (Limpeza).

Requisição 01 · Login do admin

O que faz. Autentica o administrador e devolve um token JWT.

Por que existe. O admin pode tudo que a manutenção faz e ainda escreve nos cadastros. Quase toda rota exige token, então esta é a primeira requisição a rodar. O script também prepara três valores usados nas demais pastas: um sufixo único para os nomes de teste, a data de hoje e duas matrículas aleatórias de 4 dígitos.

Método e rota	<i>POST /v1/auth/login</i>
Autenticação	Nenhuma (sem o cabeçalho Authorization)
Corpo (JSON)	<pre>{ "matricula": "0053", "senha": "123456" }</pre>
Grava no ambiente	<i>token, sufixo, hoje, matricula_a, matricula_b</i>
Resultado esperado	HTTP 200 com data.token e data.usuario (papel admin).

Requisição enviada

```
POST http://localhost:3000/v1/auth/login
```

```
{
  "matricula": "0053",
  "senha": "123456"
}
```

Resposta recebida · 200 OK (114 ms)

```
{
  "data": {
    "token": "eyJhbG... (omitido)",
    "usuario": {
      "id": 83,
      "nome": "Administrador do Sistema",
      "matricula": "0053",
      "papel": "admin"
    }
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”; “Resposta traz um token”; “Perfil é admin”.

Observações. Credenciais criadas pelo seed (npm run db:seed): matrícula 0053, senha 123456. O token dura 7 dias; se as demais requisições passarem a dar 401, rode esta de novo.

Requisição 02 · Login da manutenção

O que faz. Autentica um usuário de perfil manutenção.

Por que existe. Serve para provar, na pasta 11, que a manutenção recebe 403 ao tentar escrever nos cadastros (só o admin escreve).

Método e rota	<i>POST /v1/auth/Login</i>
Autenticação	Nenhuma (sem o cabeçalho Authorization)
Corpo (JSON)	<pre>{ "matricula": "0001", "senha": "123456" }</pre>
Grava no ambiente	<i>token_manut</i>
Resultado esperado	HTTP 200 com token e papel manutenção.

Requisição enviada

```
POST http://localhost:3000/v1/auth/login
```

```
{
  "matricula": "0001",
  "senha": "123456"
}
```

Resposta recebida · 200 OK (67 ms)

```
{
  "data": {
    "token": "eyJhbG... (omitido)",
    "usuario": {
      "id": 2,
      "nome": "Manutenção Principal",
      "matricula": "0001",
      "papel": "manutencao"
    }
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”; “Resposta traz um token”; “Perfil é manutenção”.

Observações. Matrícula 0001, senha 123456 (seed).

Requisição 03 · Obter ids de setor, categoria e atividade (auxiliar)

O que faz. Lê as listas combinadas de setores, categorias e atividades ativos e guarda o primeiro id de cada uma.

Por que existe. O cadastro de colaborador exige um setor e o de ferramenta exige uma categoria. Em vez de fixar ids que podem não existir no banco de quem testa, a coleção descobre ids válidos.

Método e rota	<i>GET /v1/opcoes</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Grava no ambiente	<i>setor_apoio_id, grupo_apoio_id, atividade_apoio_id</i>
Resultado esperado	HTTP 200 com data.setores, data.categorias e data.atividades.

Requisição enviada

```
GET http://localhost:3000/v1/opcoes
```

Resposta recebida · 200 OK (52 ms)

```
{
  "data": {
    "setores": [
      {
        "id": 4,
        "nome": "Controle de Qualidade"
      },
      {
        "id": 5,
        "nome": "Estamparia"
      },
      {
        "id": 713,
        "nome": "Manutenção Geral"
      },
      {
        "id": 248,
        "nome": "Manutenção Geral"
      },
      {
        "id": 3,
        "nome": "Montagem Industrial"
      },
      ... (resposta resumida: 182 linhas omitidas)
    ]
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Trazem setores, categorias e atividades”.

Observações. Só lê; não altera nada.

Requisição 04 · Criar colaborador de apoio (POST)

O que faz. Cadastra o colaborador que fará as retiradas das pastas 7 e 8.

Por que existe. Empréstimo exige um colaborador ativo. Criar um próprio evita depender do seed e evita mexer em dados reais.

Método e rota	<i>POST /v1/colaboradores</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Apoio Entrega {{ _.sufixo }}", "matricula": "{{ _.matricula_a }}", "setorId": {{ _.setor_apoio_id }} }</pre>
Grava no ambiente	<i>colaborador_apoio_id</i>
Resultado esperado	HTTP 201 com o colaborador criado.

Requisição enviada

POST http://localhost:3000/v1/colaboradores

```
{
  "nome": "Apoio Entrega muxc3dwc",
  "matricula": "9163",
  "setorId": 4
}
```

Resposta recebida · 201 Created (35 ms)

```
{
  "data": {
    "id": 2228,
    "nome": "Apoio Entrega muxc3dwc",
    "matricula": "9163",
    "setor_id": 4,
    "ativo": true,
    "criado_por": 83,
    "created_at": "2026-10-06T23:52:57.512Z",
    "updated_at": "2026-10-06T23:52:57.512Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Colaborador criado e ativo”.

Observações. A matrícula é aleatória (4 dígitos). Se o banco já tiver essa matrícula a API responde 409; basta rodar de novo a requisição de login (01) para sortear outra.

Requisição 05 · Criar ferramenta de apoio 1 (POST)

O que faz. Cadastra a ferramenta usada no empréstimo normal da pasta 7.

Por que existe. Cada empréstimo precisa de uma ferramenta disponível. Uma ferramenta própria garante que ela esteja livre.

Método e rota	<i>POST /v1/ferramentas</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Apoio 1 Entrega {{ _.sufixo }}", "grupoId": {{ _.grupo_apoio_id }}, "setorId": {{ _.setor_apoio_id }} }</pre>
Grava no ambiente	<i>ferramenta_apoio1_id</i>
Resultado esperado	HTTP 201 com a ferramenta criada, status disponível.

Requisição enviada

```
POST http://localhost:3000/v1/ferramentas
```

```
{
  "nome": "Apoio 1 Entrega muxc3dwc",
  "grupoId": 4,
  "setorId": 4
}
```

Resposta recebida · 201 Created (16 ms)

```
{
  "data": {
    "id": 3885,
    "nome": "Apoio 1 Entrega muxc3dwc",
    "descricao": null,
    "marca": null,
    "modelo": null,
    "codigo_identificacao": 82,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": null,
    "status": "disponivel",
    "motivo_indisponivel": null,
    "etiqueta_impressa_em": null,
    "foto_url": null,
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.536Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”, “Ferramenta criada e disponível”.

Observações. O código de identificação (1 a 9999) é gerado pelo banco.

Requisição 06 · Criar ferramenta de apoio 2 (POST)

O que faz. Cadastra a ferramenta usada na devolução com avaria da pasta 8.

Por que existe. A avaria move a ferramenta para indisponível; por isso ela precisa ser uma ferramenta de teste, separada da primeira.

Método e rota	<i>POST /v1/ferramentas</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Apoio 2 Entrega {{ _.sufixo }}", "grupoId": {{ _.grupo_apoio_id }}, "setorId": {{ _.setor_apoio_id }} }</pre>
Grava no ambiente	<i>ferramenta_apoio2_id</i>
Resultado esperado	HTTP 201 com a ferramenta criada, status disponível.

Requisição enviada

```
POST http://localhost:3000/v1/ferramentas
```

```
{
  "nome": "Apoio 2 Entrega muxc3dwc",
  "grupoId": 4,
  "setorId": 4
}
```

Resposta recebida · 201 Created (6 ms)

```
{
  "data": {
    "id": 3886,
    "nome": "Apoio 2 Entrega muxc3dwc",
    "descricao": null,
    "marca": null,
    "modelo": null,
    "codigo_identificacao": 83,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": null,
    "status": "disponivel",
    "motivo_indisponivel": null,
    "etiqueta_impressa_em": null,
    "foto_url": null,
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.549Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”, “Ferramenta criada e disponível”.

5.2 2 · Setores (tabela setores)

CRUD completo de setores: POST, GET, PUT, PATCH e DELETE (baixa lógica). Só o admin escreve.

Requisição 07 · Cadastrar setor (POST)

O que faz. Cadastra o setor novo.

Por que existe. É o "C" (Create) do CRUD de setores. Os testes seguintes da pasta usam o registro criado aqui (variável `setor_id`).

Método e rota	<i>POST /v1/setores</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Setor CRUD {{ _.sufixo }}" }</pre>
Grava no ambiente	<i>setor_id</i>
Resultado esperado	HTTP 201 com o registro criado, ativo.

Requisição enviada

```
POST http://localhost:3000/v1/setores
```

```
{
  "nome": "Setor CRUD muxc3dwc"
}
```

Resposta recebida · 201 Created (16 ms)

```
{
  "data": {
    "id": 763,
    "nome": "Setor CRUD muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.567Z",
    "updated_at": "2026-10-06T23:52:57.567Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): "Status HTTP 201", "Setor criado com os dados enviados".

Observações. Nome repetido é recusado com 409, sem diferenciar maiúsculas de minúsculas (veja a pasta 11).

Requisição 08 · Consultar setor por id (GET)

O que faz. Busca o setor cadastrado pelo id.

Por que existe. É o "R" (Read). Prova que o registro foi gravado e pode ser lido por quem opera o balcão (manutenção e admin).

Método e rota	<i>GET /v1/setores/{{ _.setor_id }}</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com o setor.

Requisição enviada

```
GET http://localhost:3000/v1/setores/763
```

Resposta recebida · 200 OK (2 ms)

```
{
  "data": {
    "id": 763,
    "nome": "Setor CRUD muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.567Z",
    "updated_at": "2026-10-06T23:52:57.567Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): "Status HTTP 200"; "É o setor cadastrado".

Requisição 09 · Substituir setor (PUT)

O que faz. Envia a representação completa do setor (nome e ativo).

Por que existe. É o PUT do setor: o cliente manda todos os campos editáveis. Na API atual, PUT e PATCH usam o mesmo tratamento (campos enviados são gravados, os ausentes ficam como estão), então o PUT completo e o PATCH parcial convivem.

Método e rota	<i>PUT</i> /v1/setores/{{ <i>_setor_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Setor CRUD (PUT) {{ <i>_sufixo</i> }}", "ativo": true }</pre>
Resultado esperado	HTTP 200 com os campos substituídos.

Requisição enviada

```
PUT http://localhost:3000/v1/setores/763
```

```
{
  "nome": "Setor CRUD (PUT) muxc3dwc",
  "ativo": true
}
```

Resposta recebida · 200 OK (13 ms)

```
{
  "data": {
    "id": 763,
    "nome": "Setor CRUD (PUT) muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.567Z",
    "updated_at": "2026-10-06T23:52:57.582Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”; “Nome foi substituído”; “Continua ativo”.

Observações. Só o admin escreve. Pela convenção REST estrita o PUT exigiria todos os campos; aqui ele aceita também envio parcial. Fica registrado como observação, sem alterar a API.

Requisição 10 · Editar setor (PATCH, campo parcial)

O que faz. Altera só o nome do setor.

Por que existe. É o "U" (Update). Mandar um único campo prova que o restante permanece intacto.

Método e rota	<i>PATCH</i> /v1/setores/{{ <i>_setor_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Setor CRUD (PATCH) {{ <i>_sufixo</i> }}" }</pre>
Resultado esperado	HTTP 200 com o nome novo e ativo = true.

Requisição enviada

```
PATCH http://localhost:3000/v1/setores/763
```

```
{
  "nome": "Setor CRUD (PATCH) muxc3dwc"
}
```

Resposta recebida · 200 OK (15 ms)

```
{
  "data": {
    "id": 763,
    "nome": "Setor CRUD (PATCH) muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.567Z",
    "updated_at": "2026-10-06T23:52:57.598Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): "Status HTTP 200"; "Nome foi alterado"; "Campo não enviado (ativo) permaneceu".

Requisição 11 · Dar baixa no setor (DELETE, baixa lógica)

O que faz. Dá baixa no setor: ativo = false. O registro continua no banco.

Por que existe. É o "D" (Delete). A baixa é lógica para não quebrar o histórico de quem já usou o cadastro.

Método e rota	<i>DELETE</i> /v1/setores/{ <i>_setor_id</i> }
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com ativo = false.

Requisição enviada

```
DELETE http://localhost:3000/v1/setores/763
```

Resposta recebida · 200 OK (14 ms)

```
{
  "data": {
    "id": 763,
    "nome": "Setor CRUD (PATCH) muxc3dwc",
    "ativo": false,
    "created_at": "2026-10-06T23:52:57.567Z",
    "updated_at": "2026-10-06T23:52:57.612Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): "Status HTTP 200"; "Registro ficou inativo".

Observações. Convenção REST: DELETE costuma devolver 204 sem corpo. A API devolve 200 com o registro; a coleção segue o comportamento real.

5.3 3 · Categorias (tabela grupos_ferramentas)

CRUD completo de categorias (grupos de ferramentas): POST, GET, PUT, PATCH e DELETE (baixa lógica). Só o admin escreve.

Requisição 12 · Cadastrar categoria (POST)

O que faz. Cadastra a categoria nova.

Por que existe. É o "C" (Create) do CRUD de categorias. Os testes seguintes da pasta usam o registro criado aqui (variável `categoria_id`).

Método e rota	<i>POST /v1/categorias</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Categoria CRUD {{ _.sufixo }}" }</pre>
Grava no ambiente	<i>categoria_id</i>
Resultado esperado	HTTP 201 com o registro criado, ativo.

Requisição enviada

```
POST http://localhost:3000/v1/categorias
```

```
{
  "nome": "Categoria CRUD muxc3dwc"
}
```

Resposta recebida · 201 Created (16 ms)

```
{
  "data": {
    "id": 756,
    "nome": "Categoria CRUD muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.628Z",
    "updated_at": "2026-10-06T23:52:57.628Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Categoria criada com os dados enviados”.

Observações. Nome repetido é recusado com 409, sem diferenciar maiúsculas de minúsculas (veja a pasta 11).

Requisição 13 · Consultar categoria por id (GET)

O que faz. Busca a categoria cadastrada pelo id.

Por que existe. É o "R" (Read). Prova que o registro foi gravado e pode ser lido por quem opera o balcão (manutenção e admin).

Método e rota	<i>GET /v1/categorias/{ {_.categoria_id } }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com a categoria.

Requisição enviada

```
GET http://localhost:3000/v1/categorias/756
```

Resposta recebida · 200 OK (14 ms)

```
{
  "data": {
    "id": 756,
    "nome": "Categoria CRUD muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.628Z",
    "updated_at": "2026-10-06T23:52:57.628Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): "Status HTTP 200"; "É a categoria cadastrada".

Requisição 14 · Substituir categoria (PUT)

O que faz. Envia a representação completa da categoria (nome e ativo).

Por que existe. É o PUT da categoria: o cliente manda todos os campos editáveis. Na API atual, PUT e PATCH usam o mesmo tratamento (campos enviados são gravados, os ausentes ficam como estão), então o PUT completo e o PATCH parcial convivem.

Método e rota	<i>PUT /v1/categorias/{_categoria_id }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Categoria CRUD (PUT) {_sufixo }", "ativo": true }</pre>
Resultado esperado	HTTP 200 com os campos substituídos.

Requisição enviada

```
PUT http://localhost:3000/v1/categorias/756
```

```
{
  "nome": "Categoria CRUD (PUT) muxc3dwc",
  "ativo": true
}
```

Resposta recebida · 200 OK (16 ms)

```
{
  "data": {
    "id": 756,
    "nome": "Categoria CRUD (PUT) muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.628Z",
    "updated_at": "2026-10-06T23:52:57.659Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”; “Nome foi substituído”; “Continua ativo”.

Observações. Só o admin escreve. Pela convenção REST estrita o PUT exigiria todos os campos; aqui ele aceita também envio parcial. Fica registrado como observação, sem alterar a API.

Requisição 15 · Editar categoria (PATCH, campo parcial)

O que faz. Altera só o nome da categoria.

Por que existe. É o "U" (Update). Mandar um único campo prova que o restante permanece intacto.

Método e rota	<i>PATCH</i> /v1/categorias/{{ <i>_categoria_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Categoria CRUD (PATCH) {{ <i>_sufixo</i> }}" }</pre>
Resultado esperado	HTTP 200 com o nome novo e ativo = true.

Requisição enviada

```
PATCH http://localhost:3000/v1/categorias/756
```

```
{
  "nome": "Categoria CRUD (PATCH) muxc3dwc"
}
```

Resposta recebida · 200 OK (15 ms)

```
{
  "data": {
    "id": 756,
    "nome": "Categoria CRUD (PATCH) muxc3dwc",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.628Z",
    "updated_at": "2026-10-06T23:52:57.675Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): "Status HTTP 200"; "Nome foi alterado"; "Campo não enviado (ativo) permaneceu".

Requisição 16 · Dar baixa na categoria (DELETE, baixa lógica)

O que faz. Dá baixa na categoria: ativo = false. O registro continua no banco.

Por que existe. É o "D" (Delete). A baixa é lógica para não quebrar o histórico de quem já usou o cadastro.

Método e rota	<i>DELETE</i> /v1/categorias/{{ <i>_categoria_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com ativo = false.

Requisição enviada

```
DELETE http://localhost:3000/v1/categorias/756
```

Resposta recebida · 200 OK (14 ms)

```
{
  "data": {
    "id": 756,
    "nome": "Categoria CRUD (PATCH) muxc3dwc",
    "ativo": false,
    "created_at": "2026-10-06T23:52:57.628Z",
    "updated_at": "2026-10-06T23:52:57.689Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): "Status HTTP 200"; "Registro ficou inativo".

Observações. Convenção REST: DELETE costuma devolver 204 sem corpo. A API devolve 200 com o registro; a coleção segue o comportamento real.

5.4 4 · Atividades (tabela atividades)

CRUD completo de atividades: POST, GET, PUT, PATCH e DELETE (baixa lógica). Só o admin escreve.

Requisição 17 · Cadastrar atividade (POST)

O que faz. Cadastra a atividade nova.

Por que existe. É o "C" (Create) do CRUD de atividades. Os testes seguintes da pasta usam o registro criado aqui (variável `atividade_id`).

Método e rota	<i>POST /v1/atividades</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Atividade CRUD {{ _.sufixo }}", "descricao": "Criada pela coleção da entrega de 06.10.2026" }</pre>
Grava no ambiente	<i>atividade_id</i>
Resultado esperado	HTTP 201 com o registro criado, ativo.

Requisição enviada

```
POST http://localhost:3000/v1/atividades
```

```
{
  "nome": "Atividade CRUD muxc3dwc",
  "descricao": "Criada pela coleção da entrega de 06.10.2026"
}
```

Resposta recebida · 201 Created (15 ms)

```
{
  "data": {
    "id": 951,
    "nome": "Atividade CRUD muxc3dwc",
    "descricao": "Criada pela coleção da entrega de 06.10.2026",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.705Z",
    "updated_at": "2026-10-06T23:52:57.705Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Atividade criada com os dados enviados”.

Observações. Nome repetido é recusado com 409, sem diferenciar maiúsculas de minúsculas (veja a pasta 11).

Requisição 18 · Consultar atividade por id (GET)

O que faz. Busca a atividade cadastrada pelo id.

Por que existe. É o "R" (Read). Prova que o registro foi gravado e pode ser lido por quem opera o balcão (manutenção e admin).

Método e rota	<i>GET /v1/atividades/{ {_.atividade_id } }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com a atividade.

Requisição enviada

```
GET http://localhost:3000/v1/atividades/951
```

Resposta recebida · 200 OK (16 ms)

```
{
  "data": {
    "id": 951,
    "nome": "Atividade CRUD muxc3dwc",
    "descricao": "Criada pela coleção da entrega de 06.10.2026",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.705Z",
    "updated_at": "2026-10-06T23:52:57.705Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): "Status HTTP 200"; "É a atividade cadastrada".

Requisição 19 · Substituir atividade (PUT)

O que faz. Envia a representação completa da atividade (nome, descrição e ativo).

Por que existe. É o PUT da atividade: o cliente manda todos os campos editáveis. Na API atual, PUT e PATCH usam o mesmo tratamento (campos enviados são gravados, os ausentes ficam como estão), então o PUT completo e o PATCH parcial convivem.

Método e rota	<i>PUT /v1/atividades/{_atividade_id }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Atividade CRUD (PUT) {_sufixo }", "descricao": "Descrição substituída via PUT", "ativo": true }</pre>
Resultado esperado	HTTP 200 com os campos substituídos.

Requisição enviada

```
PUT http://localhost:3000/v1/atividades/951

{
  "nome": "Atividade CRUD (PUT) muxc3dwc",
  "descricao": "Descrição substituída via PUT",
  "ativo": true
}
```

Resposta recebida · 200 OK (15 ms)

```
{
  "data": {
    "id": 951,
    "nome": "Atividade CRUD (PUT) muxc3dwc",
    "descricao": "Descrição substituída via PUT",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.705Z",
    "updated_at": "2026-10-06T23:52:57.737Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”; “Nome foi substituído”; “Continua ativo”.

Observações. Só o admin escreve. Pela convenção REST estrita o PUT exigiria todos os campos; aqui ele aceita também envio parcial. Fica registrado como observação, sem alterar a API.

Requisição 20 · Editar atividade (PATCH, campo parcial)

O que faz. Altera só o nome da atividade.

Por que existe. É o "U" (Update). Mandar um único campo prova que o restante permanece intacto.

Método e rota	<i>PATCH</i> /v1/atividades/{{ <i>_atividade_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Atividade CRUD (PATCH) {{ <i>_sufixo</i> }}" }</pre>
Resultado esperado	HTTP 200 com o nome novo e ativo = true.

Requisição enviada

```
PATCH http://localhost:3000/v1/atividades/951
```

```
{
  "nome": "Atividade CRUD (PATCH) muxc3dwc"
}
```

Resposta recebida · 200 OK (16 ms)

```
{
  "data": {
    "id": 951,
    "nome": "Atividade CRUD (PATCH) muxc3dwc",
    "descricao": "Descrição substituída via PUT",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.705Z",
    "updated_at": "2026-10-06T23:52:57.753Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”; “Nome foi alterado”; “Campo não enviado (ativo) permaneceu”.

Requisição 21 · Dar baixa na atividade (DELETE, baixa lógica)

O que faz. Dá baixa na atividade: ativo = false. O registro continua no banco.

Por que existe. É o "D" (Delete). A baixa é lógica para não quebrar o histórico de quem já usou o cadastro.

Método e rota	<i>DELETE</i> /v1/atividades/{ <i>_atividade_id</i> }
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com ativo = false.

Requisição enviada

```
DELETE http://localhost:3000/v1/atividades/951
```

Resposta recebida · 200 OK (15 ms)

```
{
  "data": {
    "id": 951,
    "nome": "Atividade CRUD (PATCH) muxc3dwc",
    "descricao": "Descrição substituída via PUT",
    "ativo": false,
    "created_at": "2026-10-06T23:52:57.705Z",
    "updated_at": "2026-10-06T23:52:57.768Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Registro ficou inativo”.

Observações. Convenção REST: DELETE costuma devolver 204 sem corpo. A API devolve 200 com o registro; a coleção segue o comportamento real.

5.5 5 · Colaboradores (tabelas colaboradores, usuarios e convites_acesso)

POST, GET, PATCH e DELETE (inativação) de colaboradores. O PUT não existe na API (a edição é só PATCH) e é testado como rota inexistente. Inclui o fluxo do convite de acesso, que grava em convites_acesso e cria o usuário (tabela usuarios) quando o colaborador define a senha.

Requisição 22 · Cadastrar colaborador (POST)

O que faz. Cadastra um colaborador com nome, matrícula e setor.

Por que existe. É o "C" (Create) do CRUD de colaboradores. A pasta usa este colaborador nos testes seguintes.

Método e rota	<i>POST /v1/colaboradores</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Colaborador CRUD {{ _.sufixo }}", "matricula": "{{ _.matricula_b }}", "setorId": {{ _.setor_apoio_id }} }</pre>
Grava no ambiente	<i>colaborador_id</i>
Resultado esperado	HTTP 201 com o colaborador criado e ativo.

Requisição enviada

POST http://localhost:3000/v1/colaboradores

```
{
  "nome": "Colaborador CRUD muxc3dwc",
  "matricula": "3422",
  "setorId": 4
}
```

Resposta recebida · 201 Created (16 ms)

```
{
  "data": {
    "id": 2229,
    "nome": "Colaborador CRUD muxc3dwc",
    "matricula": "3422",
    "setor_id": 4,
    "ativo": true,
    "criado_por": 83,
    "created_at": "2026-10-06T23:52:57.784Z",
    "updated_at": "2026-10-06T23:52:57.784Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Colaborador criado com os dados enviados”.

Observações. Matrícula repetida é recusada (409); veja a pasta 11.

Requisição 23 · Consultar colaborador por id (GET)

O que faz. Busca o colaborador cadastrado pelo id.

Por que existe. É o "R" (Read) do CRUD. Prova que o registro foi gravado e pode ser lido.

Método e rota	<code>GET /v1/colaboradores/{{ _.colaborador_id }}</code>
Autenticação	Bearer Token = variável <code>token</code> (admin)
Resultado esperado	HTTP 200 com o colaborador.

Requisição enviada

```
GET http://localhost:3000/v1/colaboradores/2229
```

Resposta recebida · 200 OK (15 ms)

```
{
  "data": {
    "id": 2229,
    "nome": "Colaborador CRUD muxc3dwc",
    "matricula": "3422",
    "setor_id": 4,
    "ativo": true,
    "criado_por": 83,
    "created_at": "2026-10-06T23:52:57.784Z",
    "updated_at": "2026-10-06T23:52:57.784Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): "Status HTTP 200"; "É o colaborador cadastrado".

Requisição 24 · Substituir colaborador (PUT) — rota inexistente

O que faz. Tenta substituir o colaborador com PUT.

Por que existe. A API não oferece PUT em colaboradores: a edição parcial é o PATCH. Esta requisição documenta isso e prova que a rota realmente não existe, em vez de aceitar a chamada em silêncio.

Método e rota	<i>PUT /v1/colaboradores/{_colaborador_id_}</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Nome via PUT" }</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
PUT http://localhost:3000/v1/colaboradores/2229
```

```
{
  "nome": "Nome via PUT"
}
```

Resposta recebida · 404 Not Found (14 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: PUT /v1/colaboradores/2229",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”, “Código de erro NOT_FOUND”, “Mensagem diz que a rota não existe”.

Observações. Convenção REST: PUT substitui o recurso inteiro; PATCH altera campos. O projeto escolheu só PATCH (veja docs/api-07-put-para-patch.md).

Requisição 25 · Editar colaborador (PATCH, campo parcial)

O que faz. Altera só o nome do colaborador.

Por que existe. É o "U" (Update) do CRUD. Mandar um único campo prova que os demais (matrícula, setor) ficam intactos.

Método e rota	<i>PATCH</i> /v1/colaboradores/{{ <i>_colaborador_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Colaborador CRUD Editado {{ <i>_sufixo</i> }}" }</pre>
Resultado esperado	HTTP 200 com o nome novo e a matrícula original.

Requisição enviada

```
PATCH http://localhost:3000/v1/colaboradores/2229
```

```
{
  "nome": "Colaborador CRUD Editado muxc3dwc"
}
```

Resposta recebida · 200 OK (18 ms)

```
{
  "data": {
    "id": 2229,
    "nome": "Colaborador CRUD Editado muxc3dwc",
    "matricula": "3422",
    "setor_id": 4,
    "ativo": true,
    "criado_por": 83,
    "created_at": "2026-10-06T23:52:57.784Z",
    "updated_at": "2026-10-06T23:52:57.832Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”; “Nome foi alterado”; “Matrícula permaneceu a mesma”.

Observações. Edição é só do admin; a manutenção recebe 403.

Requisição 26 · Gerar convite de acesso (POST)

O que faz. Gera o link de acesso do colaborador (grava na tabela `convites_acesso`).

Por que existe. O colaborador cadastrado pelo admin recebe um link para definir a própria senha. O token só aparece nesta resposta (o banco guarda o hash).

Método e rota	<i>POST /v1/colaboradores/{ { _colaborador_id } }/convite</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Grava no ambiente	<i>token_convite</i>
Resultado esperado	HTTP 201 com token, <i>expiraEm</i> e o colaborador.

Requisição enviada

```
POST http://localhost:3000/v1/colaboradores/2229/convite
```

Resposta recebida · 201 Created (20 ms)

```
{
  "data": {
    "token": "plu4_V... (omitido)",
    "expiraEm": "2026-10-13T23:52:57.847Z",
    "colaborador": {
      "id": 2229,
      "nome": "Colaborador CRUD Editado muxc3dwc",
      "matricula": "3422"
    }
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Resposta traz token de 43 caracteres”.

Observações. O convite vale 7 dias e um único uso.

Requisição 27 · Consultar convite (GET, rota pública)

O que faz. Consulta o convite sem estar logado, como faz a tela que o colaborador abre pelo link.

Por que existe. Confirma que o convite é válido antes de pedir a senha e que a rota pública não vazava dados além do necessário.

Método e rota	<i>GET /v1/auth/convites/{{ _.token_convite }}</i>
Autenticação	Nenhuma (sem o cabeçalho Authorization)
Resultado esperado	HTTP 200 com o nome do colaborador.

Requisição enviada

```
GET http://localhost:3000/v1/auth/convites/plu4_Vc-EI3MIch8RBcldXdIg01GXpBa8rPm0bccgCk
```

Resposta recebida · 200 OK (9 ms)

```
{
  "data": {
    "nome": "Colaborador CRUD Editado muxc3dwc",
    "matricula": "3422"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”, “Convite válido”.

Observações. Rota pública, com limite de requisições.

Requisição 28 · Aceitar convite e definir senha (POST, cria usuário)

O que faz. O colaborador define a própria senha de 6 dígitos. A API cria o usuário (tabela usuarios) e devolve um token de sessão.

Por que existe. Cobre as tabelas usuarios e convites_acesso: o convite é consumido e passa a existir um usuário com o papel manutenção.

Método e rota	<i>POST /v1/auth/convites/{{ _.token_convite }}/senha</i>
Autenticação	Nenhuma (sem o cabeçalho Authorization)
Corpo (JSON)	<pre>{ "senha": "246810" }</pre>
Resultado esperado	HTTP 200 ou 201 com token de sessão.

Requisição enviada

```
POST http://localhost:3000/v1/auth/convites/plu4_Vc-EI3MIch8RBcldXdIg01GXpBa8rPmObccgCk/senha

{
  "senha": "246810"
}
```

Resposta recebida · 200 OK (77 ms)

```
{
  "data": {
    "token": "eyJhbG... (omitido)",
    "usuario": {
      "id": 1040,
      "nome": "Colaborador CRUD Editado muxc3dwc",
      "matricula": "3422",
      "papel": "manutencao"
    }
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200 ou 201”; “Resposta traz token de sessão”.

Observações. O convite só funciona uma vez; repetir esta requisição devolve erro.

Requisição 29 · Inativar colaborador (DELETE, baixa lógica)

O que faz. Inativa o colaborador (ativo = false). O registro continua no banco.

Por que existe. É o "D" (Delete). A exclusão é lógica para preservar o histórico de empréstimos e ocorrências desse colaborador.

Método e rota	<i>DELETE</i> /v1/colaboradores/{ <i>_colaborador_id</i> }
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com o colaborador inativo.

Requisição enviada

```
DELETE http://localhost:3000/v1/colaboradores/2229
```

Resposta recebida · 200 OK (16 ms)

```
{
  "data": {
    "id": 2229,
    "nome": "Colaborador CRUD Editado muxc3dwc",
    "matricula": "3422",
    "setor_id": 4,
    "ativo": false,
    "criado_por": 83,
    "created_at": "2026-10-06T23:52:57.784Z",
    "updated_at": "2026-10-06T23:52:57.956Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Colaborador ficou inativo”.

Observações. Convenção REST: DELETE costuma devolver 204 sem corpo. A API devolve 200 com o registro; a coleção segue o comportamento real.

5.6 6 · Ferramentas (tabela ferramentas)

POST, GET, PATCH e DELETE (baixa lógica) de ferramentas. O PUT da ferramenta é o envio da foto (PUT /ferramentas/:id/foto).

Requisição 30 · Cadastrar ferramenta (POST)

O que faz. Cadastra uma ferramenta nova.

Por que existe. É o "C" (Create) do CRUD de ferramentas.

Método e rota	POST /v1/ferramentas
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Furadeira CRUD {{ _.sufixo }}", "marca": "Bosch", "grupoId": {{ _.grupo_apoio_id }}, "setorId": {{ _.setor_apoio_id }}, "localizacao": "Prateleira A" }</pre>
Grava no ambiente	<i>ferramenta_id</i>
Resultado esperado	HTTP 201 com a ferramenta criada, status disponível e código de identificação gerado.

Requisição enviada

```
POST http://localhost:3000/v1/ferramentas
```

```
{
  "nome": "Furadeira CRUD muxc3dwc",
  "marca": "Bosch",
  "grupoId": 4,
  "setorId": 4,
  "localizacao": "Prateleira A"
}
```

Resposta recebida · 201 Created (17 ms)

```
{
  "data": {
    "id": 3887,
    "nome": "Furadeira CRUD muxc3dwc",
    "descricao": null,
    "marca": "Bosch",
    "modelo": null,
    "codigo_identificacao": 84,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": "Prateleira A",
    "status": "disponivel",
    "motivo_indisponivel": null,
    "etiqueta_impressa_em": null,
    "foto_url": null,
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.971Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Ferramenta criada e disponível”.

Observações. O `codigo_identificacao` (1 a 9999) é gerado pelo banco, não é enviado.

Requisição 31 · Consultar ferramenta por id (GET)

O que faz. Busca a ferramenta cadastrada pelo id.

Por que existe. É o "R" (Read). Prova que o registro foi gravado com os dados enviados.

Método e rota	<code>GET /v1/ferramentas/{_ferramenta_id }</code>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com a ferramenta.

Requisição enviada

```
GET http://localhost:3000/v1/ferramentas/3887
```

Resposta recebida · 200 OK (13 ms)

```
{
  "data": {
    "id": 3887,
    "nome": "Furadeira CRUD muxc3dwc",
    "descricao": null,
    "marca": "Bosch",
    "modelo": null,
    "codigo_identificacao": 84,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": "Prateleira A",
    "status": "disponivel",
    "motivo_indisponivel": null,
    "etiqueta_impressa_em": null,
    "foto_url": null,
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.971Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”, “Marca e localização conferem”.

Requisição 32 · Enviar foto da ferramenta (PUT /foto)

O que faz. Envia a imagem da ferramenta. O PUT grava (ou substitui) a foto e guarda o endereço em foto_url.

Por que existe. É o PUT da ferramenta: o recurso /foto é substituído por inteiro a cada envio, que é o uso correto do PUT.

Método e rota	<i>PUT /v1/ferramentas/{ { _ferramenta_id } }/foto</i>
Autenticação	Bearer Token = variável token (admin)
Corpo (image/webp)	RIFF0000WEBPVP8 CRUD-06-10-2026
Resultado esperado	HTTP 200 com foto_url preenchido.

Requisição enviada

```
PUT http://localhost:3000/v1/ferramentas/3887/foto
```

```
RIFF0000WEBPVP8 CRUD-06-10-2026
```

Resposta recebida · 200 OK (18 ms)

```
{
  "data": {
    "id": 3887,
    "nome": "Furadeira CRUD muxc3dwc",
    "descricao": null,
    "marca": "Bosch",
    "modelo": null,
    "codigo_identificacao": 84,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": "Prateleira A",
    "status": "disponivel",
    "motivo_indisponivel": null,
    "etiqueta_impressa_em": null,
    "foto_url": "/uploads/3887-f87cf2b3-8b62-41b1-aba3-0e66e6a941a4.webp",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.971Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “foto_url foi preenchido”.

Observações. Para não depender de um arquivo no seu computador, o corpo é um texto mínimo com o cabeçalho de um arquivo WebP (RIFF....WEBP), que é o que a API confere. Para testar com uma imagem de verdade, abra a aba Body, escolha "Binary File" e selecione um JPEG, PNG ou WebP de até 5 MB.

Requisição 33 · Editar ferramenta (PATCH, campos parciais)

O que faz. Altera só a marca e a localização da ferramenta.

Por que existe. É o "U" (Update). Poucos campos de propósito: o nome e os demais devem ficar intactos.

Método e rota	<i>PATCH</i> /v1/ferramentas/{ <i>_ferramenta_id</i> }
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "marca": "Makita", "localizacao": "Prateleira B" }</pre>
Resultado esperado	HTTP 200 com marca e localização novas e o nome original.

Requisição enviada

```
PATCH http://localhost:3000/v1/ferramentas/3887
```

```
{
  "marca": "Makita",
  "localizacao": "Prateleira B"
}
```

Resposta recebida · 200 OK (13 ms)

```
{
  "data": {
    "id": 3887,
    "nome": "Furadeira CRUD muxc3dwc",
    "descricao": null,
    "marca": "Makita",
    "modelo": null,
    "codigo_identificacao": 84,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": "Prateleira B",
    "status": "disponivel",
    "motivo_indisponivel": null,
    "etiqueta_impressa_em": null,
    "foto_url": "/uploads/3887-f87cf2b3-8b62-41b1-aba3-0e66e6a941a4.webp",
    "ativo": true,
    "created_at": "2026-10-06T23:52:57.971Z"
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 200”, “Campos enviados foram alterados”; “Nome permaneceu igual”.

Observações. Edição é só do admin.

Requisição 34 · Dar baixa na ferramenta (DELETE, baixa lógica)

O que faz. Dá baixa na ferramenta: ativo = false, status indisponível e motivo baixada.

Por que existe. É o "D" (Delete). A baixa é lógica para preservar o histórico de empréstimos.

Método e rota	<i>DELETE /v1/ferramentas/{{ _.ferramenta_id }}</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com a ferramenta baixada.

Requisição enviada

```
DELETE http://localhost:3000/v1/ferramentas/3887
```

Resposta recebida · 200 OK (18 ms)

```
{
  "data": {
    "id": 3887,
    "nome": "Furadeira CRUD muxc3dwc",
    "descricao": null,
    "marca": "Makita",
    "modelo": null,
    "codigo_identificacao": 84,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": "Prateleira B",
    "status": "indisponivel",
    "motivo_indisponivel": "baixada",
    "etiqueta_impressa_em": null,
    "foto_url": "/uploads/3887-f87cf2b3-8b62-41b1-aba3-0e66e6a941a4.webp",
    "ativo": false,
    "created_at": "2026-10-06T23:52:57.971Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Ferramenta ficou inativa e baixada”.

Observações. Ferramenta com empréstimo em aberto não pode ser baixada (409). Convenção REST: a API devolve 200 com o registro, não 204.

5.7 7 · Empréstimos (tabela emprestimos)

Retirada (POST), consulta (GET) e devolução (PATCH). PUT e DELETE não existem por regra de negócio: o empréstimo é histórico e nunca é apagado nem substituído. A devolução é um PATCH em /devolucao.

Requisição 35 · Registrar retirada (POST)

O que faz. Registra a retirada da ferramenta de apoio 1 pelo colaborador de apoio. A previsão de devolução é hoje (até 23:59, horário de Brasília).

Por que existe. É o "C" (Create) do empréstimo. A ferramenta passa a em_uso e o responsável pelo registro vem do usuário logado (JWT), nunca do corpo (Regra 6).

Método e rota	<i>POST /v1/emprestimos</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "ferramentaId": {{ _.ferramenta_apoio1_id }}, "colaboradorId": {{ _.colaborador_apoio_id }}, "setorDestinoId": {{ _.setor_apoio_id }}, "atividadeId": {{ _.atividade_apoio_id }}, "previsaoDevolucao": "{{ _.hoje }}" }</pre>
Grava no ambiente	<i>emprestimo_id</i>
Resultado esperado	HTTP 201 com o empréstimo aberto.

Requisição enviada

POST http://localhost:3000/v1/emprestimos

```
{
  "ferramentaId": 3885,
  "colaboradorId": 2228,
  "setorDestinoId": 4,
  "atividadeId": 10,
  "previsaoDevolucao": "2026-10-06"
}
```

Resposta recebida · 201 Created (21 ms)

```
{
  "data": {
    "id": 1915,
    "data_retirada": "2026-10-06T23:52:58.050Z",
    "previsao_devolucao": "2026-10-07T02:59:59.000Z",
    "data_devolucao": null,
    "condicao_devolucao": null,
    "ordem_servico": null,
    "observacoes_retirada": null,
    "observacoes_devolucao": null,
    "situacao": "em_aberto",
    "ferramenta_id": 3885,
    "ferramenta_nome": "Apoio 1 Entrega muxc3dwc",
    "codigo_identificacao": 82,
    "eh_kit": false,
    "item_kit_id": null,
    "item_kit_nome": null,
    "colaborador_id": 2228,
    "colaborador_nome": "Apoio Entrega muxc3dwc",
    "colaborador_matricula": "9163",
  }
}
```

```
"setor_id": 4,
"setor_nome": "Controle de Qualidade",
"atividade_id": 10,
"atividade_nome": "Apoio de Linha",
... (resposta resumida: 6 linhas omitidas)
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Empréstimo criado e aberto”.

Observações. O banco permite um único empréstimo aberto por ferramenta (índice único parcial, Regra 1). Se rodar depois das 23:59 a previsão cai no passado e a API responde 400.

Requisição 36 · Listar empréstimos em aberto (GET)

O que faz. Lista os empréstimos em aberto, com paginação.

Por que existe. É o "R" (Read). O empréstimo criado na requisição anterior precisa aparecer na lista.

Método e rota	<i>GET /v1/emprestimos?situacao=em_aberto&limit=100</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com data (lista) e meta (paginação), incluindo o empréstimo criado.

Requisição enviada

```
GET http://localhost:3000/v1/emprestimos?situacao=em_aberto&limit=100
```

Resposta recebida · 200 OK (20 ms)

```
{
  "data": [
    {
      "id": 1915,
      "data_retirada": "2026-10-06T23:52:58.050Z",
      "previsao_devolucao": "2026-10-07T02:59:59.000Z",
      "data_devolucao": null,
      "condicao_devolucao": null,
      "ordem_servico": null,
      "observacoes_retirada": null,
      "observacoes_devolucao": null,
      "situacao": "em_aberto",
      "ferramenta_id": 3885,
      "ferramenta_nome": "Apoio 1 Entrega muxc3dwc",
      "codigo_identificacao": 82,
      "eh_kit": false,
      "item_kit_id": null,
      "item_kit_nome": null,
      "colaborador_id": 2228,
      "colaborador_nome": "Apoio Entrega muxc3dwc",
      "colaborador_matricula": "9163",
      "setor_id": 4,
      "setor_nome": "Controle de Qualidade",
      "atividade_id": 10,
      ... (resposta resumida: 14 linhas omitidas)
    }
  ]
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Lista contém o empréstimo criado”.

Observações. A API não tem GET por id para empréstimo; a consulta é a lista com filtros.

Requisição 37 · Substituir empréstimo (PUT) — rota inexistente

O que faz. Tenta substituir o empréstimo com PUT.

Por que existe. O empréstimo é um registro histórico: a única mudança permitida é a devolução, que tem rota própria. Substituí-lo por inteiro apagaria o que aconteceu.

Método e rota	<i>PUT /v1/emprestimos/{ {_.emprestimo_id } }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ }</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
PUT http://localhost:3000/v1/emprestimos/1915
```

```
{ }
```

Resposta recebida · 404 Not Found (17 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: PUT /v1/emprestimos/1915",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

Requisição 38 · Registrar devolução (PATCH /devolucao)

O que faz. Registra a devolução da ferramenta em perfeito estado.

Por que existe. É o "U" (Update) do empréstimo: fecha o registro gravando data_devolucao e condicao_devolucao e libera a ferramenta (volta a disponível).

Método e rota	<i>PATCH /v1/emprestimos/{ { _emprestimo_id } }/devolucao</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "condicaoDevolucao": "ok", "observacaoDevolucao": "Devolvida sem problemas (teste)" }</pre>
Resultado esperado	HTTP 200 com data_devolucao preenchida e condição ok.

Requisição enviada

PATCH http://localhost:3000/v1/emprestimos/1915/devolucao

```
{
  "condicaoDevolucao": "ok",
  "observacaoDevolucao": "Devolvida sem problemas (teste)"
}
```

Resposta recebida · 200 OK (24 ms)

```
{
  "data": {
    "id": 1915,
    "data_retirada": "2026-10-06T23:52:58.050Z",
    "previsao_devolucao": "2026-10-07T02:59:59.000Z",
    "data_devolucao": "2026-10-06T23:52:58.113Z",
    "condicao_devolucao": "ok",
    "ordem_servico": null,
    "observacoes_retirada": null,
    "observacoes_devolucao": "Devolvida sem problemas (teste)",
    "situacao": "devolvido",
    "ferramenta_id": 3885,
    "ferramenta_nome": "Apoio 1 Entrega muxc3dwc",
    "codigo_identificacao": 82,
    "eh_kit": false,
    "item_kit_id": null,
    "item_kit_nome": null,
    "colaborador_id": 2228,
    "colaborador_nome": "Apoio Entrega muxc3dwc",
    "colaborador_matricula": "9163",
    "setor_id": 4,
    "setor_nome": "Controle de Qualidade",
    "atividade_id": 10,
    "atividade_nome": "Apoio de Linha",
    ... (resposta resumida: 7 linhas omitidas)
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”, “Devolução registrada”.

Observações. Avaria ou perda na devolução abrem uma ocorrência automaticamente (pasta 8).

Requisição 39 · Apagar empréstimo (DELETE) — rota inexistente

O que faz. Tenta apagar o empréstimo.

Por que existe. Empréstimo nunca é apagado: o histórico de retiradas e devoluções é a base das ocorrências, notificações e relatórios.

Método e rota	<i>DELETE</i> /v1/emprestimos/{{ <i>_emprestimo_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
DELETE http://localhost:3000/v1/emprestimos/1915
```

Resposta recebida · 404 Not Found (7 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: DELETE /v1/emprestimos/1915",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

5.8 8 · Ocorrências (tabela ocorrencias)

A ocorrência não é criada por POST: ela nasce sozinha quando uma devolução registra avaria ou perda (Regra 3). A pasta provoca esse caminho e então consulta e atualiza a ocorrência. POST, PUT e DELETE diretos são testados como rotas inexistentes.

Requisição 40 · Criar ocorrência direto (POST) — rota inexistente

O que faz. Tenta criar uma ocorrência por POST.

Por que existe. A ocorrência só nasce de uma devolução com avaria ou perda (Regra 3), herdando o colaborador responsável. Se existisse POST direto, daria para criar ocorrência sem empréstimo.

Método e rota	<i>POST /v1/ocorrencias</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "tipo": "AVARIA" }</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
POST http://localhost:3000/v1/ocorrencias

{
  "tipo": "AVARIA"
}
```

Resposta recebida · 404 Not Found (15 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: POST /v1/ocorrencias",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

Observações. As próximas duas requisições provocam o caminho correto.

Requisição 41 · Retirar a ferramenta de apoio 2 (auxiliar)

O que faz. Registra um segundo empréstimo, com a ferramenta de apoio 2.

Por que existe. É o passo preparatório: para gerar uma ocorrência é preciso ter um empréstimo aberto para devolver com avaria.

Método e rota	<i>POST /v1/emprestimos</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "ferramentaId": {{ _.ferramenta_apoio2_id }}, "colaboradorId": {{ _.colaborador_apoio_id }}, "setorDestinoId": {{ _.setor_apoio_id }}, "previsaoDevolucao": "{{ _.hoje }}" }</pre>
Grava no ambiente	<i>emprestimo2_id</i>
Resultado esperado	HTTP 201 com o empréstimo aberto.

Requisição enviada

```
POST http://localhost:3000/v1/emprestimos
```

```
{
  "ferramentaId": 3886,
  "colaboradorId": 2228,
  "setorDestinoId": 4,
  "previsaoDevolucao": "2026-10-06"
}
```

Resposta recebida · 201 Created (21 ms)

```
{
  "data": {
    "id": 1916,
    "data_retirada": "2026-10-06T23:52:58.159Z",
    "previsao_devolucao": "2026-10-07T02:59:59.000Z",
    "data_devolucao": null,
    "condicao_devolucao": null,
    "ordem_servico": null,
    "observacoes_retirada": null,
    "observacoes_devolucao": null,
    "situacao": "em_aberto",
    "ferramenta_id": 3886,
    "ferramenta_nome": "Apoio 2 Entrega muxc3dwc",
    "codigo_identificacao": 83,
    "eh_kit": false,
    "item_kit_id": null,
    "item_kit_nome": null,
    "colaborador_id": 2228,
    "colaborador_nome": "Apoio Entrega muxc3dwc",
    "colaborador_matricula": "9163",
    "setor_id": 4,
    "setor_nome": "Controle de Qualidade",
    "atividade_id": null,
  }
}
```

```
"atividade_nome": null,  
... (resposta resumida: 6 linhas omitidas)
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 201”; “Empréstimo criado”.

Observações. Requisição auxiliar.

Requisição 42 · Devolver com avaria — abre a ocorrência (PATCH)

O que faz. Devolve a ferramenta de apoio 2 com avaria.

Por que existe. Prova a Regra 3: a avaria abre sozinha uma ocorrência (tabela ocorrencias) herdando o colaborador e move a ferramenta para indisponível.

Método e rota	<i>PATCH /v1/emprestimos/{{ _ .emprestimo2_id }}/devolucao</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "condicaoDevolucao": "avaria", "observacaoDevolucao": "Cabo danificado (teste)", "custoEstimado": 80 }</pre>
Resultado esperado	HTTP 200 com condição avaria.

Requisição enviada

```
PATCH http://localhost:3000/v1/emprestimos/1916/devolucao
```

```
{
  "condicaoDevolucao": "avaria",
  "observacaoDevolucao": "Cabo danificado (teste)",
  "custoEstimado": 80
}
```

Resposta recebida · 200 OK (16 ms)

```
{
  "data": {
    "id": 1916,
    "data_retirada": "2026-10-06T23:52:58.159Z",
    "previsao_devolucao": "2026-10-07T02:59:59.000Z",
    "data_devolucao": "2026-10-06T23:52:58.173Z",
    "condicao_devolucao": "avaria",
    "ordem_servico": null,
    "observacoes_retirada": null,
    "observacoes_devolucao": "Cabo danificado (teste)",
    "situacao": "devolvido",
    "ferramenta_id": 3886,
    "ferramenta_nome": "Apoio 2 Entrega muxc3dwc",
    "codigo_identificacao": 83,
    "eh_kit": false,
    "item_kit_id": null,
    "item_kit_nome": null,
    "colaborador_id": 2228,
    "colaborador_nome": "Apoio Entrega muxc3dwc",
    "colaborador_matricula": "9163",
    "setor_id": 4,
    "setor_nome": "Controle de Qualidade",
    "atividade_id": null,
    "atividade_nome": null,
    ... (resposta resumida: 7 linhas omitidas)
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Devolução com avaria registrada”.

Observações. O "POST" da ocorrência acontece aqui, por dentro da API.

Requisição 43 · Listar ocorrências abertas (GET)

O que faz. Lista as ocorrências abertas do colaborador de apoio e guarda o id da que acabou de nascer.

Por que existe. É o "R" (Read). Confirma que a avaria virou uma ocorrência aberta, ligada ao empréstimo e ao colaborador.

Método e rota	<i>GET /v1/ocorrencias?status=aberta&colaboradorId={{ _.colaborador_apoio_id }}&limit=100</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Grava no ambiente	<i>ocorrencia_id</i>
Resultado esperado	HTTP 200 com a ocorrência do empréstimo de apoio 2, tipo AVARIA, status aberta.

Requisição enviada

```
GET http://localhost:3000/v1/ocorrencias?status=aberta&colaboradorId=2228&limit=100
```

Resposta recebida · 200 OK (12 ms)

```
{
  "data": [
    {
      "id": 1494,
      "emprestimo_id": 1916,
      "ferramenta_id": 3886,
      "ferramenta_nome": "Apoio 2 Entrega muxc3dwc",
      "item_kit_id": null,
      "item_kit_nome": null,
      "colaborador_id": 2228,
      "colaborador_nome": "Apoio Entrega muxc3dwc",
      "colaborador_matricula": "9163",
      "tipo": "AVARIA",
      "descricao": "Cabo danificado (teste)",
      "status": "aberta",
      "custo_estimado": "80.00",
      "custo_real": null,
      "data_resolucao": null,
      "observacoes_resolucao": null,
      "registrada_por": 83,
      "registrada_por_nome": "Administrador do Sistema",
      "resolvida_por": null,
      "resolvida_por_nome": null,
      "created_at": "2026-10-06T23:52:58.173Z",
      ... (resposta resumida: 10 linhas omitidas)
    }
  ]
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”, “Existe a ocorrência do empréstimo devolvido com avaria”.

Requisição 44 · Substituir ocorrência (PUT) — rota inexistente

O que faz. Tenta substituir a ocorrência com PUT.

Por que existe. A ocorrência avança por etapas (aberta, em reparo, cobrada, resolvida, baixada) e só tem PATCH, para que ninguém reescreva o histórico de uma vez.

Método e rota	<i>PUT /v1/ocorrencias/{ {_.ocorrencia_id } }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ }</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
PUT http://localhost:3000/v1/ocorrencias/1494
```

```
{ }
```

Resposta recebida · 404 Not Found (12 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: PUT /v1/ocorrencias/1494",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

Requisição 45 · Atualizar ocorrência (PATCH)

O que faz. Move a ocorrência para em_reparo e registra o custo estimado.

Por que existe. É o "U" (Update) da ocorrência.

Método e rota	<i>PATCH /v1/ocorrencias/{_ .ocorrencia_id }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "status": "em_reparo", "custoEstimado": 120.5, "observacoesResolucao": "Enviada para reparo (teste)" }</pre>
Resultado esperado	HTTP 200 com status em_reparo.

Requisição enviada

```
PATCH http://localhost:3000/v1/ocorrencias/1494

{
  "status": "em_reparo",
  "custoEstimado": 120.5,
  "observacoesResolucao": "Enviada para reparo (teste)"
}
```

Resposta recebida · 200 OK (20 ms)

```
{
  "data": {
    "id": 1494,
    "emprestimo_id": 1916,
    "ferramenta_id": 3886,
    "ferramenta_nome": "Apoio 2 Entrega muxc3dwc",
    "item_kit_id": null,
    "item_kit_nome": null,
    "colaborador_id": 2228,
    "colaborador_nome": "Apoio Entrega muxc3dwc",
    "colaborador_matricula": "9163",
    "tipo": "AVARIA",
    "descricao": "Cabo danificado (teste)",
    "status": "em_reparo",
    "custo_estimado": "120.50",
    "custo_real": null,
    "data_resolucao": null,
    "observacoes_resolucao": "Enviada para reparo (teste)",
    "registrada_por": 83,
    "registrada_por_nome": "Administrador do Sistema",
    "resolvida_por": null,
    "resolvida_por_nome": null,
    "created_at": "2026-10-06T23:52:58.173Z",
    "updated_at": "2026-10-06T23:52:58.220Z",
    ... (resposta resumida: 3 linhas omitidas)
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”, “Status mudou para em_reparo”.

Requisição 46 · Apagar ocorrência (DELETE) — rota inexistente

O que faz. Tenta apagar a ocorrência.

Por que existe. Ocorrência é registro de prejuízo e responsabilidade; encerra-se (resolvida ou baixada), nunca é apagada.

Método e rota	<i>DELETE</i> /v1/ocorrencias/{{ <i>_ocorrencia_id</i> }}
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
DELETE http://localhost:3000/v1/ocorrencias/1494
```

Resposta recebida · 404 Not Found (11 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: DELETE /v1/ocorrencias/1494",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

5.9 9 · Notificações (tabela notificacoes)

As notificações são geradas pelo sistema (rotina diária das 07:00, seg a sex) para empréstimos atrasados ou que vencem hoje. A API só lista e marca como lida. POST, PUT e DELETE são testados como rotas inexistentes.

Requisição 47 · Criar notificação (POST) — rota inexistente

O que faz. Tenta criar uma notificação por POST.

Por que existe. Notificação é gerada pelo sistema a partir dos empréstimos atrasados ou que vencem hoje; ninguém a cria à mão.

Método e rota	<i>POST /v1/notificacoes</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "titulo": "x" }</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
POST http://localhost:3000/v1/notificacoes
```

```
{
  "titulo": "x"
}
```

Resposta recebida · 404 Not Found (1 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: POST /v1/notificacoes",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

Requisição 48 · Listar notificações (GET)

O que faz. Lista as notificações e guarda o id de uma ainda não lida (ou 999999 se não houver nenhuma).

Por que existe. É o "R" (Read). O resultado depende do dia: pode vir vazia, e isso também é uma resposta correta.

Método e rota	<i>GET /v1/notificacoes?limit=100</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Grava no ambiente	<i>notificacao_id</i>
Resultado esperado	HTTP 200 com data (lista) e meta (paginação).

Requisição enviada

```
GET http://localhost:3000/v1/notificacoes?limit=100
```

Resposta recebida · 200 OK (19 ms)

```
{
  "data": [
    {
      "id": 60,
      "tipo": "atraso",
      "titulo": "Empréstimo atrasado",
      "mensagem": "ZZINSOMNIA Devolucao H com Carlos Eduardo Souza está atrasada desde 01/10.",
      "lida": true,
      "link": "/ferramentas/2421?emprestimo=1272",
      "created_at": "2026-10-06T23:52:22.844Z"
    },
    {
      "id": 59,
      "tipo": "atraso",
      "titulo": "Empréstimo atrasado",
      "mensagem": "Furadeira de Impacto Bosch GSB 13 RE com Carlos Eduardo Souza está atrasada desde 26/09.",
      "lida": false,
      "link": "/ferramentas/10?emprestimo=1",
      "created_at": "2026-10-06T23:52:22.844Z"
    },
    {
      "id": 58,
      "tipo": "atraso",
      "titulo": "Empréstimo atrasado",
      ... (resposta resumida: 148 linhas omitidas)
    }
  ]
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”, “Resposta é uma lista com paginação”.

Observações. A rotina que gera as notificações roda de segunda a sexta às 07:00 (Brasília).

Requisição 49 · Substituir notificação (PUT) — rota inexistente

O que faz. Tenta substituir uma notificação com PUT.

Por que existe. A única mudança permitida numa notificação é marcá-la como lida.

Método e rota	<i>PUT /v1/notificacoes/{_ .notificacao_id }</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{}</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
PUT http://localhost:3000/v1/notificacoes/59

{}
```

Resposta recebida · 404 Not Found (10 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: PUT /v1/notificacoes/59",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

Requisição 50 · Marcar notificação como lida (PATCH)

O que faz. Marca como lida a notificação escolhida na listagem.

Por que existe. É o "U" (Update). Se a listagem trouxe uma notificação não lida, ela é marcada (200); se veio vazia, o id usado é 999999 e a API responde 404, o que prova o tratamento de id inexistente.

Método e rota	<i>PATCH /v1/notificacoes/{_notificacao_id}/Lida</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 (notificação existente) ou 404 (id 999999).

Requisição enviada

```
PATCH http://localhost:3000/v1/notificacoes/59/lida
```

Resposta recebida · 200 OK (17 ms)

```
{
  "data": {
    "id": 59,
    "tipo": "atraso",
    "titulo": "Empréstimo atrasado",
    "mensagem": "Furadeira de Impacto Bosch GSB 13 RE com Carlos Eduardo Souza está atrasada desde 26/09.",
    "lida": true,
    "link": "/ferramentas/10?emprestimo=1",
    "created_at": "2026-10-06T23:52:22.844Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200 ou 404”; “Status confere com o id usado”.

Observações. O teste aceita os dois resultados e confere que o status bate com o id usado.

Requisição 51 · Apagar notificação (DELETE) — rota inexistente

O que faz. Tenta apagar uma notificação.

Por que existe. Notificações lidas são limpas pela própria rotina do sistema; a API não expõe exclusão.

Método e rota	<i>DELETE</i> /v1/notificacoes/{ <i>_notificacao_id</i> }
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
DELETE http://localhost:3000/v1/notificacoes/59
```

Resposta recebida · 404 Not Found (13 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: DELETE /v1/notificacoes/59",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

5.10 10 · Feriados (tabela feriados)

Os feriados vêm da BrasilAPI e ficam em cache na tabela. A API sincroniza (POST /feriados/sincronizar) e consulta (GET). PUT, PATCH e DELETE são testados como rotas inexistentes.

Requisição 52 · Sincronizar feriados com a BrasilAPI (POST)

O que faz. Busca os feriados nacionais do ano atual na BrasilAPI e grava (ou atualiza) na tabela feriados.

Por que existe. É o "C" (Create) dos feriados. Cobre a integração com a fonte externa. Se a BrasilAPI estiver fora, a API usa o fallback de sábado e domingo (Regra 9).

Método e rota	<i>POST /v1/feriados/sincronizar</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com a lista de feriados do ano.

Requisição enviada

```
POST http://localhost:3000/v1/feriados/sincronizar
```

Resposta recebida · 200 OK (115 ms)

```
{
  "data": {
    "ano": 2026,
    "fonte": "brasil_api",
    "feriados": [
      {
        "id": 1,
        "data": "2026-01-01",
        "nome": "Confraternização mundial",
        "tipo": "national",
        "ano": 2026
      },
      {
        "id": 2,
        "data": "2026-02-16",
        "nome": "Carnaval",
        "tipo": "national",
        "ano": 2026
      },
      {
        "id": 3,
        "data": "2026-02-17",
        "nome": "Carnaval",
        "tipo": "national",
        "ano": 2026
      }
    ]
  }
}
... (resposta resumida: 82 linhas omitidas)
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Resposta traz dados”.

Observações. Precisa de internet para a BrasilAPI; sem ela a API cai no fallback e o teste tolera isso.

Requisição 53 · Consultar feriados do ano (GET)

O que faz. Lista os feriados do ano atual a partir do cache em tabela.

Por que existe. É o "R" (Read). Rota pública: o calendário do front consulta sem login.

Método e rota	<i>GET /v1/feriados</i>
Autenticação	Nenhuma (sem o cabeçalho Authorization)
Resultado esperado	HTTP 200 com data.feriados.

Requisição enviada

```
GET http://localhost:3000/v1/feriados
```

Resposta recebida · 200 OK (10 ms)

```
{
  "data": {
    "ano": 2026,
    "fonte": "cache",
    "feriados": [
      {
        "id": 1,
        "data": "2026-01-01",
        "nome": "Confraternização mundial",
        "tipo": "national",
        "ano": 2026
      },
      {
        "id": 2,
        "data": "2026-02-16",
        "nome": "Carnaval",
        "tipo": "national",
        "ano": 2026
      },
      {
        "id": 3,
        "data": "2026-02-17",
        "nome": "Carnaval",
        "tipo": "national",
        ... (resposta resumida: 82 linhas omitidas)
      }
    ]
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Traz a lista de feriados do ano”.

Requisição 54 · Substituir feriado (PUT) — rota inexistente

O que faz. Tenta substituir um feriado com PUT.

Por que existe. Feriado vem da fonte oficial (BrasilAPI) e não é editado à mão.

Método e rota	<i>PUT /v1/feriados/1</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "x" }</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
PUT http://localhost:3000/v1/feriados/1
```

```
{
  "nome": "x"
}
```

Resposta recebida · 404 Not Found (13 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: PUT /v1/feriados/1",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

Requisição 55 · Editar feriado (PATCH) — rota inexistente

O que faz. Tenta editar um feriado com PATCH.

Por que existe. Mesma razão do PUT: o dado é da fonte oficial. A única forma de atualizá-lo é sincronizar de novo.

Método e rota	<i>PATCH /v1/feriados/1</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "x" }</pre>
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
PATCH http://localhost:3000/v1/feriados/1
```

```
{
  "nome": "x"
}
```

Resposta recebida · 404 Not Found (16 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: PATCH /v1/feriados/1",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

Requisição 56 · Apagar feriado (DELETE) — rota inexistente

O que faz. Tenta apagar um feriado.

Por que existe. Apagar um feriado mudaria o cálculo de dias úteis das previsões de devolução.

Método e rota	<i>DELETE /v1/feriados/1</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 404 NOT_FOUND ("Rota não encontrada").

Requisição enviada

```
DELETE http://localhost:3000/v1/feriados/1
```

Resposta recebida · 404 Not Found (15 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Rota não encontrada: DELETE /v1/feriados/1",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Mensagem diz que a rota não existe”.

5.11 11 · Validação e tratamento de erros (400, 401, 403, 404, 409)

Provam que a API recusa o que é inválido com o envelope de erro padrão { error: { code, message, details } }. Nenhuma altera o banco.

Requisição 57 · Criar setor sem token — 401

O que faz. Tenta criar um setor sem enviar o token.

Por que existe. Prova que a rota é protegida.

Método e rota	<i>POST /v1/setores</i>
Autenticação	Nenhuma (sem o cabeçalho Authorization)
Corpo (JSON)	<pre>{ "nome": "Sem Token" }</pre>
Resultado esperado	HTTP 401 TOKEN_NOT_PROVIDED no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/setores
```

```
{
  "nome": "Sem Token"
}
```

Resposta recebida · 401 Unauthorized (3 ms)

```
{
  "error": {
    "code": "TOKEN_NOT_PROVIDED",
    "message": "Token de autenticação não fornecido",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 401”; “Código de erro TOKEN_NOT_PROVIDED”; “Resposta no envelope de erro padrão”.

Requisição 58 · Criar setor com token inválido — 401

O que faz. Tenta criar um setor com um token adulterado.

Por que existe. Prova que a API valida a assinatura do JWT, não só a presença.

Método e rota	<i>POST /v1/setores</i>
Autenticação	Bearer Token adulterado, de propósito
Corpo (JSON)	<pre>{ "nome": "Token Invalido" }</pre>
Resultado esperado	HTTP 401 TOKEN_INVALID no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/setores
```

```
{
  "nome": "Token Invalido"
}
```

Resposta recebida · 401 Unauthorized (13 ms)

```
{
  "error": {
    "code": "TOKEN_INVALID",
    "message": "Token de autenticação inválido",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 401”; “Código de erro TOKEN_INVALID”; “Resposta no envelope de erro padrão”.

Requisição 59 · Criar setor como manutenção — 403

O que faz. A manutenção tenta criar um setor.

Por que existe. Cadastros auxiliares só são escritos pelo admin (Regra 8); a manutenção só consulta.

Método e rota	<i>POST /v1/setores</i>
Autenticação	Bearer Token = variável <i>token_manut</i> (manutenção)
Corpo (JSON)	<pre>{ "nome": "Sem Permissao" }</pre>
Resultado esperado	HTTP 403 ACCESS_DENIED no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/setores
```

```
{
  "nome": "Sem Permissao"
}
```

Resposta recebida · 403 Forbidden (16 ms)

```
{
  "error": {
    "code": "ACCESS_DENIED",
    "message": "Perfil 'manutencao' não possui permissão para acessar esta rota",
    "details": [
      {
        "requiredRoles": [
          "admin"
        ],
        "userRole": "manutencao"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 403”; “Código de erro ACCESS_DENIED”; “Resposta no envelope de erro padrão”.

Requisição 60 · Editar colaborador como manutenção — 403

O que faz. A manutenção tenta editar um colaborador.

Por que existe. Edição e inativação de colaborador são só do admin.

Método e rota	<i>PATCH</i> /v1/colaboradores/{ <i>_colaborador_apoio_id</i> }
Autenticação	Bearer Token = variável <i>token_manut</i> (manutenção)
Corpo (JSON)	<pre>{ "nome": "Sem Permissao" }</pre>
Resultado esperado	HTTP 403 ACCESS_DENIED no envelope { error: { code, message, details } }.

Requisição enviada

```
PATCH http://localhost:3000/v1/colaboradores/2228
```

```
{
  "nome": "Sem Permissao"
}
```

Resposta recebida · 403 Forbidden (16 ms)

```
{
  "error": {
    "code": "ACCESS_DENIED",
    "message": "Perfil 'manutencao' não possui permissão para acessar esta rota",
    "details": [
      {
        "requiredRoles": [
          "admin"
        ],
        "userRole": "manutencao"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 403”; “Código de erro ACCESS_DENIED”; “Resposta no envelope de erro padrão”.

Requisição 61 · Criar setor com nome vazio — 400

O que faz. Cria um setor com nome vazio.

Por que existe. A validação (Zod) recusa antes de chegar ao banco.

Método e rota	<i>POST /v1/setores</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "" }</pre>
Resultado esperado	HTTP 400 VALIDATION_ERROR no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/setores
```

```
{
  "nome": ""
}
```

Resposta recebida · 400 Bad Request (15 ms)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Erro de validação nos dados enviados",
    "details": [
      {
        "field": "nome",
        "message": "Nome não pode ser vazio",
        "rule": "too_small"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 400”; “Código de erro VALIDATION_ERROR”; “Resposta no envelope de erro padrão”.

Requisição 62 · Criar ferramenta sem grupold — 400

O que faz. Cadastra uma ferramenta sem a categoria (grupold), que é obrigatória.

Por que existe. Campo obrigatório ausente.

Método e rota	<i>POST /v1/ferramentas</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Sem Grupo" }</pre>
Resultado esperado	HTTP 400 VALIDATION_ERROR no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/ferramentas
```

```
{
  "nome": "Sem Grupo"
}
```

Resposta recebida · 400 Bad Request (16 ms)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Erro de validação nos dados enviados",
    "details": [
      {
        "field": "grupoId",
        "message": "grupoId deve ser um número",
        "rule": "invalid_type"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 400”; “Código de erro VALIDATION_ERROR”; “Resposta no envelope de erro padrão”.

Requisição 63 · Criar colaborador com matrícula inválida — 400

O que faz. Cadastra colaborador com matrícula que não tem 4 dígitos numéricos.

Por que existe. A matrícula tem formato fixo (0001 a 9999).

Método e rota	<i>POST /v1/colaboradores</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Matricula Ruim", "matricula": "ABCD", "setorId": 1 }</pre>
Resultado esperado	HTTP 400 VALIDATION_ERROR no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/colaboradores
```

```
{
  "nome": "Matricula Ruim",
  "matricula": "ABCD",
  "setorId": 1
}
```

Resposta recebida · 400 Bad Request (15 ms)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Erro de validação nos dados enviados",
    "details": [
      {
        "field": "matricula",
        "message": "Matrícula deve ter exatamente 4 dígitos numéricos (0001 a 9999)",
        "rule": "invalid_string"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 400”, “Código de erro VALIDATION_ERROR”, “Resposta no envelope de erro padrão”.

Requisição 64 · Buscar setor com id não numérico — 400

O que faz. Busca um setor com id "abc".

Por que existe. O parâmetro de rota também é validado.

Método e rota	GET /v1/setores/abc
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 400 VALIDATION_ERROR no envelope { error: { code, message, details } }.

Requisição enviada

```
GET http://localhost:3000/v1/setores/abc
```

Resposta recebida · 400 Bad Request (16 ms)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Erro de validação nos dados enviados",
    "details": [
      {
        "field": "id",
        "message": "ID deve ser um número",
        "rule": "invalid_type"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 400”; “Código de erro VALIDATION_ERROR”; “Resposta no envelope de erro padrão”.

Requisição 65 · Buscar setor inexistente — 404

O que faz. Busca um setor que não existe.

Por que existe. A API responde 404 em vez de 200 vazio ou erro 500.

Método e rota	<i>GET</i> <code>/v1/setores/999999</code>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 404 NOT_FOUND no envelope { error: { code, message, details } }.

Requisição enviada

```
GET http://localhost:3000/v1/setores/999999
```

Resposta recebida · 404 Not Found (16 ms)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Setor não encontrado",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro NOT_FOUND”; “Resposta no envelope de erro padrão”.

Requisição 66 · Buscar ferramenta inexistente — 404

O que faz. Busca uma ferramenta que não existe.

Por que existe. Código de erro específico da entidade.

Método e rota	<i>GET /v1/ferramentas/999999</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 404 FERRAMENTA_NOT_FOUND no envelope { error: { code, message, details } }.

Requisição enviada

```
GET http://localhost:3000/v1/ferramentas/999999
```

Resposta recebida · 404 Not Found (15 ms)

```
{
  "error": {
    "code": "FERRAMENTA_NOT_FOUND",
    "message": "Ferramenta não encontrada",
    "details": []
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 404”; “Código de erro FERRAMENTA_NOT_FOUND”; “Resposta no envelope de erro padrão”.

Requisição 67 · Criar setor com nome repetido — 409

O que faz. Cria um setor com um nome que já existe (Usinagem CNC, do seed).

Por que existe. O índice único do banco recusa nomes repetidos, sem diferenciar maiúsculas de minúsculas, e a API traduz o erro do PostgreSQL para 409.

Método e rota	<i>POST /v1/setores</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Usinagem CNC" }</pre>
Resultado esperado	HTTP 409 DUPLICATE_ENTRY no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/setores
```

```
{
  "nome": "Usinagem CNC"
}
```

Resposta recebida · 409 Conflict (17 ms)

```
{
  "error": {
    "code": "DUPLICATE_ENTRY",
    "message": "Registro duplicado ou operação em conflito com o estado atual.",
    "details": [
      {
        "detail": "Chave (lower(nome::text))=(usinagem cnc) já existe.",
        "constraint": "uq_setores_nome_lower"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 409”; “Código de erro DUPLICATE_ENTRY”; “Resposta no envelope de erro padrão”.

Requisição 68 · Criar colaborador com matrícula repetida — 409

O que faz. Cadastra um colaborador com a matrícula do colaborador de apoio (já em uso).

Por que existe. Matrícula é única.

Método e rota	<i>POST /v1/colaboradores</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Corpo (JSON)	<pre>{ "nome": "Matricula Repetida", "matricula": "{{ _.matricula_a }}", "setorId": {{ _.setor_apoio_id }} }</pre>
Resultado esperado	HTTP 409 DUPLICATE_ENTRY no envelope { error: { code, message, details } }.

Requisição enviada

```
POST http://localhost:3000/v1/colaboradores
```

```
{
  "nome": "Matricula Repetida",
  "matricula": "9163",
  "setorId": 4
}
```

Resposta recebida · 409 Conflict (15 ms)

```
{
  "error": {
    "code": "DUPLICATE_ENTRY",
    "message": "Registro duplicado ou operação em conflito com o estado atual.",
    "details": [
      {
        "detail": "Chave (matricula)=(9163) já existe.",
        "constraint": "colaboradores_matricula_key"
      }
    ]
  }
}
```

Testes automáticos. 3/3 aprovados (PASS): “Status HTTP 409”; “Código de erro DUPLICATE_ENTRY”; “Resposta no envelope de erro padrão”.

5.12 12 · Limpeza dos dados de apoio

Dá baixa nas duas ferramentas de apoio e inativa o colaborador de apoio criados na pasta 1. Fica por último porque as pastas 7 e 8 usam esses dados.

Requisição 69 · Dar baixa na ferramenta de apoio 1

O que faz. Dá baixa na ferramenta de apoio 1.

Por que existe. Remove da operação os dados de teste criados na pasta 1.

Método e rota	<i>DELETE /v1/ferramentas/{{ _.ferramenta_apoio1_id }}</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com a ferramenta baixada.

Requisição enviada

```
DELETE http://localhost:3000/v1/ferramentas/3885
```

Resposta recebida · 200 OK (16 ms)

```
{
  "data": {
    "id": 3885,
    "nome": "Apoio 1 Entrega muxc3dwc",
    "descricao": null,
    "marca": null,
    "modelo": null,
    "codigo_identificacao": 82,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": null,
    "status": "indisponivel",
    "motivo_indisponivel": "baixada",
    "etiqueta_impressa_em": null,
    "foto_url": null,
    "ativo": false,
    "created_at": "2026-10-06T23:52:57.536Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Ferramenta baixada”.

Observações. Só funciona porque a ferramenta foi devolvida na pasta 7 (sem empréstimo aberto).

Requisição 70 · Dar baixa na ferramenta de apoio 2

O que faz. Dá baixa na ferramenta de apoio 2.

Por que existe. Idem. Ela já estava indisponível por causa da avaria; a baixa registra o motivo baixada.

Método e rota	<i>DELETE /v1/ferramentas/{{ _.ferramenta_apoio2_id }}</i>
Autenticação	Bearer Token = variável <i>token</i> (admin)
Resultado esperado	HTTP 200 com a ferramenta baixada.

Requisição enviada

```
DELETE http://localhost:3000/v1/ferramentas/3886
```

Resposta recebida · 200 OK (13 ms)

```
{
  "data": {
    "id": 3886,
    "nome": "Apoio 2 Entrega muxc3dwc",
    "descricao": null,
    "marca": null,
    "modelo": null,
    "codigo_identificacao": 83,
    "grupo_id": 4,
    "subgrupo_id": null,
    "setor_id": 4,
    "localizacao": null,
    "status": "indisponivel",
    "motivo_indisponivel": "baixada",
    "etiqueta_impressa_em": null,
    "foto_url": null,
    "ativo": false,
    "created_at": "2026-10-06T23:52:57.549Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”, “Ferramenta baixada”.

Requisição 71 · Inativar o colaborador de apoio

O que faz. Inativa o colaborador de apoio.

Por que existe. Remove da operação o colaborador de teste. O histórico dos empréstimos dele é preservado.

Método e rota	DELETE /v1/colaboradores/{ { _colaborador_apoio_id } }
Autenticação	Bearer Token = variável token (admin)
Resultado esperado	HTTP 200 com o colaborador inativo.

Requisição enviada

```
DELETE http://localhost:3000/v1/colaboradores/2228
```

Resposta recebida · 200 OK (15 ms)

```
{
  "data": {
    "id": 2228,
    "nome": "Apoio Entrega muxc3dwc",
    "matricula": "9163",
    "setor_id": 4,
    "ativo": false,
    "criado_por": 83,
    "created_at": "2026-10-06T23:52:57.512Z",
    "updated_at": "2026-10-06T23:52:58.685Z"
  }
}
```

Testes automáticos. 2/2 aprovados (PASS): “Status HTTP 200”; “Colaborador inativo”.

6. Validações e tratamento de erros

Toda recusa da API segue o mesmo envelope definido no projeto, { **"error": { "code", "message", "details" } }** }, para que o front trate os erros de um jeito só. A pasta 11 exercita cada família de erro:

HTTP	Código	Quando acontece	Requisição
401	TOKEN_NOT_PROVIDED	Tenta criar um setor sem enviar o token.	57
401	TOKEN_INVALID	Tenta criar um setor com um token adulterado.	58
403	ACCESS_DENIED	A manutenção tenta criar um setor.	59
403	ACCESS_DENIED	A manutenção tenta editar um colaborador.	60
400	VALIDATION_ERROR	Cria um setor com nome vazio.	61
400	VALIDATION_ERROR	Cadastra uma ferramenta sem a categoria (grupold), que é obrigatória.	62
400	VALIDATION_ERROR	Cadastra colaborador com matrícula que não tem 4 dígitos numéricos.	63

400	VALIDATION_ERROR	Busca um setor com id "abc".	64
404	NOT_FOUND	Busca um setor que não existe.	65
404	FERRAMENTA_NOT_FOUND	Busca uma ferramenta que não existe.	66
409	DUPLICATE_ENTRY	Cria um setor com um nome que já existe (Usinagem CNC, do seed).	67
409	DUPLICATE_ENTRY	Cadastra um colaborador com a matrícula do colaborador de apoio (já em uso).	68

Além dessas, as pastas de CRUD provam outros tratamentos: 404 para rota inexistente (**NOT_FOUND**) nas operações que a API não oferece, 409 **FERRAMENTA_COM_EMPRESTIMO_ABERTO** ao dar baixa em ferramenta emprestada (coberto na coleção da entrega de 15/09/2026, req. 14).

6.1 Validação em duas camadas

- **Na API (Zod):** tipo, tamanho, formato (matrícula de 4 dígitos, ids numéricos positivos, datas) e campos obrigatórios. Responde 400 **VALIDATION_ERROR** com a lista de campos em *details*.
- **No banco (PostgreSQL):** índices únicos (nome de setor/categoria/atividade sem diferenciar maiúsculas, matrícula, um empréstimo aberto por ferramenta) e chaves estrangeiras. Violações viram 409 **DUPLICATE_ENTRY**.

7. Observações e pontos de atenção

- **PUT x PATCH nos cadastros auxiliares.** Em setores, categorias e atividades o PUT e o PATCH usam hoje o mesmo tratamento: os campos enviados são gravados e os ausentes ficam como estão. Pela convenção REST estrita, o PUT exigiria todos os campos. A coleção envia o PUT completo e o PATCH parcial; a diferença de rigor fica registrada aqui, sem alterar a API.
- **DELETE devolve 200 com o registro**, não 204 sem corpo. A coleção segue o comportamento real da API.
- **Rotas inexistentes por regra de negócio.** As requisições "rota inexistente" são intencionais: provam que empréstimo, ocorrência, notificação e feriado não podem ser apagados ou reescritos, o que protege o histórico e a auditoria.
- **Dados de teste.** Os DELETES são baixas lógicas, então os cadastros criados pelos testes continuam no banco como inativos. Em um banco compartilhado, prefira rodar a coleção no ambiente de desenvolvimento local.
- **Previsão de devolução.** A requisição 35 usa a data de hoje (válida até 23:59, horário de Brasília). Executada depois desse horário, a API responde 400 por previsão no passado.
- **Matrículas de teste.** São sorteadas (4 dígitos). Se a sorteada já existir no banco, o POST responde 409; basta rodar de novo a requisição 01.
- **Foto da ferramenta.** A requisição 32 envia um corpo mínimo com o cabeçalho de um arquivo WebP, que é o que a API confere. Para testar com uma imagem real, use a aba Body → Binary File.

8. Conclusão

As funcionalidades CRUD estão implementadas e testadas para todas as entidades que a API expõe, com validações em duas camadas e tratamento de erros padronizado. A coleção do Insomnia reproduz o ciclo completo (criar, consultar, substituir, editar e dar baixa), prova as regras de negócio que impedem certas operações e limpa os dados de apoio ao final. Das 71 requisições, 71 passaram em todos os testes na execução registrada.